

0.27.2. Le carré avec deplace — doublé, deux positions

Les deux : le A à gauche (départ (0, 0)), puis le B à droite (départ (9, 0)).

1. Ce qu'il faut comprendre

C'est le 0.27 et le 0.27.1 réunis : les dix lignes du A, puis les dix du B.

Les deux trajets ne se touchent pas : le A reste dans les colonnes 0 à 10, le B dans les colonnes 9 à 19, sur d'autres cases.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace>` → [0.25.3. La fonction deplace\(\) — avancer sur les deux axes](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1416 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      69 o ← la console, à l'allumage - la mise en route de la console
deplace()      236 o ← main() - une fonction fournie par le langage
main()         289 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()   × 2   26 o sur place, dans deplace() + AdresseCase (10 o)
effacer() × 1   15 o sur place, dans deplace() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 2    6 o sur place, dans deplace(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace() × 10  222 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace> // fait avancer une tuile sur les deux axes
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne du A
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;      // sa ligne
```

4. UNE VARIABLE — XB

```
uint8_t xb = 9;      // la colonne du B
```

5. UNE VARIABLE — YB

```
uint8_t yb = 0;      // sa ligne
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {          // Le programme commence ici.
    // deplace(colonne, ligne, tuile, pasX, pasY) : les deux axes à la fois.
    // 1. Le A (ALPHABET[0]), depuis (0, 0) :      pasX pasY
    deplace(x, y, ALPHABET[0], 5, 5);              // la diagonale : (5, 5)
    deplace(x, y, ALPHABET[0], 5, 0);              // à droite      : (10, 5)
    deplace(x, y, ALPHABET[0], 0, 5);              // en bas        : (10, 10)
    deplace(x, y, ALPHABET[0], -5, 0);             // à gauche       : (5, 10)
    deplace(x, y, ALPHABET[0], 0, -5);             // en haut        : (5, 5)
    // 2. Le B (ALPHABET[1]), depuis (9, 0), avec ses variables xb et yb :
    deplace(xb, yb, ALPHABET[1], 5, 5);            // (14, 5)
    deplace(xb, yb, ALPHABET[1], 5, 0);            // (19, 5)
    deplace(xb, yb, ALPHABET[1], 0, 5);            // (19, 10)
    deplace(xb, yb, ALPHABET[1], -5, 0);           // (14, 10)
    deplace(xb, yb, ALPHABET[1], 0, -5);           // (14, 5)

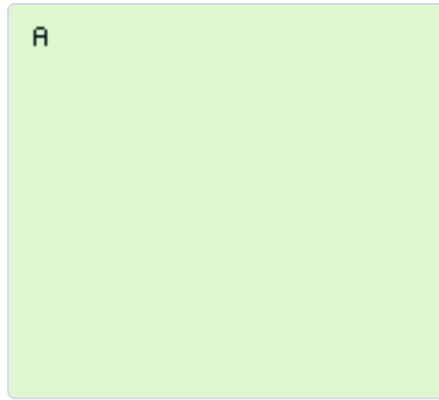
    while (true) {    // La boucle de jeu, pour toujours :
        image();      // attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

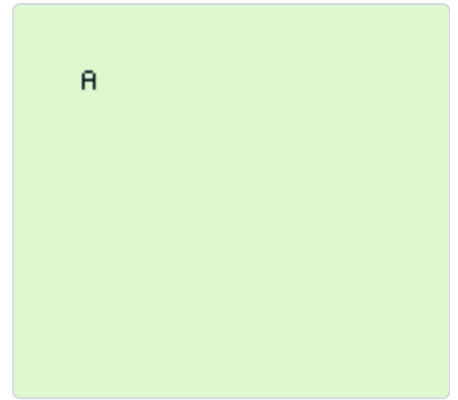
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



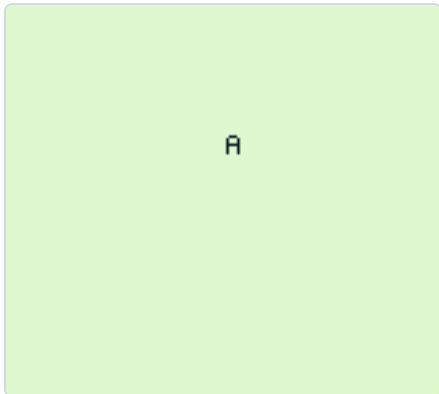
la toute première image — la console vient de démarrer — image 1



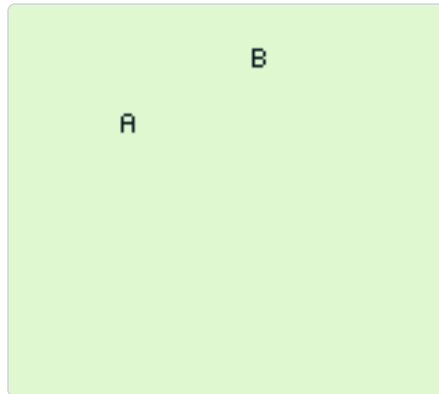
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A fait sa diagonale et son carré à gauche, puis le B à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (5, 5) (5,5)

✓ le B finit en (14, 5) (14,5)

Ensuite : **0.27.3. La fonction `va_a()` — aller jusqu'à une case** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++